Problem 1: Singly Linked List – In-Place Reversal & Pointer Tracing

Given a singly linked list: head -> [10] -> [25] -> [40] -> [15] -> NULL.

- **Task 1:** Write a function/algorithm in pseudocode to reverse the singly linked list, including three pointers (prev, curr, next) to complete your function.
- **Task 2:** Draw a diagram or step-by-step pointer trace table showing the positions and values of the three pointers (prev, curr, next) after each iteration until the list is fully reversed.

NOTE: DO NOT USE ALGORITHMS LIBRARY.

Problem 2: Circular Linked List – Insert at Head

Given a singly circular linked list: head -> [10] -> [20] -> [30] -> (points back to head).

- **Task 1:** Write pseudocode for the function InsertHead(head, newValue) to insert a new node containing newValue at the front of the circular linked list while maintaining its circular structure.
- **Task 2:** Trace the execution steps (via text description or pointer diagrams) when inserting newValue = 5 into the given list.

Problem 3: Doubly Linked List – Insert Before Target & Edge Cases (2.5 Points)

Given a doubly linked list containing integers.

- **Task 1:** Write pseudocode for InsertBefore(head, target, newValue) to insert a new node containing newValue immediately before the first occurrence of target.
- **Task 2:** List the exact order of pointer assignments required and briefly justify why this specific sequence avoids memory leaks or broken links.
- **Task 3:** Explicitly handle the following two edge cases in code/pseudocode:
    1. target is located at the head node (head).
    2. target does not exist in the list.

Problem 4: Sorted Linked List – In-Order Insertion

Given a singly linked list sorted in ascending order: head -> [2] -> [5] -> [8] -> NULL.

- **Task 1:** Write the function SortedInsert(head, val) to insert val into the list such that the ascending order is preserved.
- **Task 2:** Construct a step-by-step trace table detailing pointer iterations and condition evaluations when inserting val = 6 into the list.

NOTE: DO NOT USE ALGORITHMS LIBRARY.